Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 6

Disciplina: Tehnici de programare

Tema: ” Forensics & Stenografie ”

A efectuat : st. gr.SI – 264 Danuța Daniel

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

Scopul lucrării de laborator constă în studierea și aplicarea metodelor de analiză forensic și steganografie, utilizând limbajul de programare Python.

În cadrul lucrării se urmărește identificarea și decodarea informațiilor codificate în formatele Base64, Hex și URL, decriptarea mesajelor prin metoda XOR și identificarea parolelor pe baza hash-urilor MD5.

De asemenea, lucrarea presupune analizarea structurii fișierelor prin intermediul semnăturilor binare, extragerea metadatelor EXIF și a coordonatelor GPS din imagini, precum și identificarea și recuperarea arhivelor ascunse prin tehnica file carving. O altă sarcină importantă constă în implementarea metodei de steganografie LSB pentru ascunderea și extragerea mesajelor din imagini digitale.

Mersul Lucrarii

Structura:

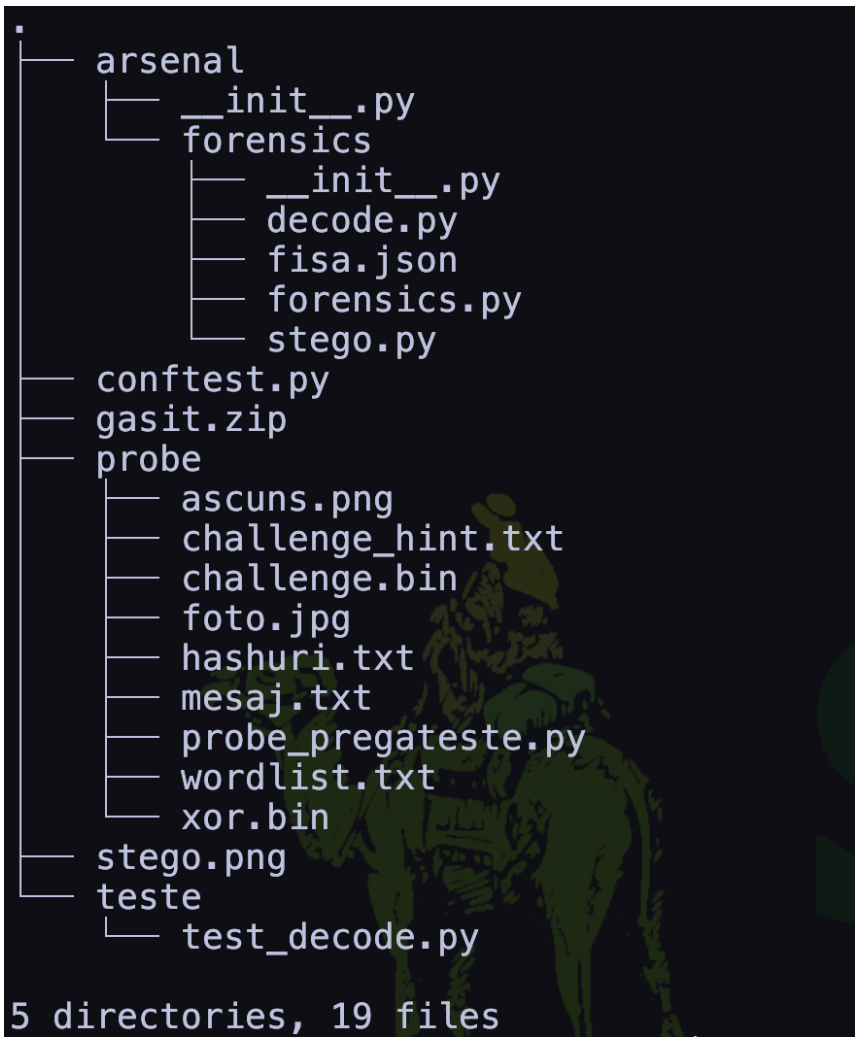


Fig 1.1-Structura lab6

1.Lucrarea A – Decoder Universal(decode.py)

```python
# A1

s = "danu"
b = s.encode()

# print(b, b[0])
# print(b.hex())
# print(bytes.fromhex(b.hex()))

# A2

def ghici_strat(s):
    if re.search(r"%[0-9A-Fa-f]{2}", s):
        return "url"

    if len(s) % 2 == 0 and re.fullmatch(r"[0-9A-Fa-f]+", s):
        return "hex"

    if len(s) % 4 == 0 and re.fullmatch(r"[A-Za-z0-9+/]+={0,2}", s):
        return "base64"

    return "necunoscut"


# print(ghici_strat("%2Fetc"))
# print(ghici_strat("48656c6c6f"))
# print(ghici_strat("U2FsdXQ="))
```

Fig 2.1 – A1/A2

```python

# A3

def desfa(s):
    strat = ghici_strat(s)

    if strat == "base64":
        rezultat = base64.b64decode(s)

    elif strat == "hex":
        rezultat = bytes.fromhex(s)

    elif strat == "url":
        rezultat = unquote(s).encode()

    else:
        rezultat = s.encode()

    return strat, rezultat


# print(desfa("U2FsdXQ="))
# print(desfa("48656c6c6f"))
# print(desfa("%2Fetc"))


# A4

def citibil(b):
    return all(32 <= c < 127 for c in b)


val = "NTM1OTU3NTk="

for _ in range(8):
    strat = ghici_strat(val)

    if strat == "necunoscut":
        break

    strat, b = desfa(val)

    if not citibil(b):
        break

    val = b.decode()

# print(val)

```

Fig 2.2 – A3/A4

```python

    # A5

    date = open("probe/xor.bin", "rb").read()

    def sparge_xor(date):
        gasite = []
        for k in range(256):
            clar = bytes(b ^ k for b in date)
            if citibil(clar):
                gasite.append((k, clar.decode()))
        return gasite


    # --------- FLAG{xor_un_octet} ------------


    # A6

    import hashlib
    tinta = open("probe/hashuri.txt").readline().strip()
    def sparge_hash(tinta, cale_wordlist="probe/wordlist.txt"):
        for cuv in open(cale_wordlist, encoding="latin1"):
            cuv = cuv.strip()
            if hashlib.md5(cuv.encode()).hexdigest() == tinta:
                return cuv
        return None

    # 32
    # gasit: password

```

Fig 2.3 – A5/A6

```python
# A7

import argparse
import json

def desfa_straturi(val, max_pasi=8):
    straturi = []
    for _ in range(max_pasi):
        if ghici_strat(val) == "necunoscut":
            break
        strat, b = desfa(val)
        if not citibil(b):
            break
        straturi.append(strat)
        val = b.decode()
    return straturi, val

if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("intrare")
    p.add_argument("--fisier", action="store_true")
    a = p.parse_args()

    if a.fisier:
        val = open(a.intrare, "rb").read().decode().strip()
    else:
        val = a.intrare

    straturi, rezultat = desfa_straturi(val)
    print(straturi, rezultat)

    fisa = {"strat": straturi, "rezultat": rezultat}
    json.dump(fisa, open("arsenal/forensics/fisa.json", "w"), indent=2)

# {
#   "strat": [
#     "base64",
#     "base64",
#     "hex"
#   ],
#   "rezultat": "FLAG{straturi_de_encoding}"
# }
```

Fig 2.4 – A7

2.Lucrarea B – Forensics și steganografie(forensics.py/stego.py)

```
6     # B1
7
8     def tip_real(cale):
9         cap = open(cale, "rb").read(8)
10        if cap.startswith(b"%PDF"):
11            return "pdf"
12        if cap.startswith(b"\xff\xd8"):
13            return "jpeg"
14        if cap.startswith(b"PK\x03\x04"):
15            return "jpeg"
16        return cap.hex()
17
18
19    # B2
20
21    # date = open("probe/ascuns.png", "rb").read()
22    # for m in re.findall(rb"[ -~]{4,}", date):
23    #     print(m.decode())
24
```

Fig 3.1 – B1/B2

```python
# B3

def arata_exif(cale):
    exif = Image.open(cale)._getexif() or {}
    for id, val in exif.items():
        print(TAGS.get(id, id), "=", val)


def dms_in_zecimal(dms, emisfera):
    grade, minute, secunde = dms
    zecimal = float(grade) + float(minute) / 60 + float(secunde) / 3600
    if emisfera in ("S", "W"):
        zecimal = -zecimal
    return zecimal

def gps_zecimal(cale):
    exif = Image.open(cale)._getexif() or {}
    gps = exif.get(34853)
    if not gps:
        return None
    lat = dms_in_zecimal(gps[2], gps[1])
    lon = dms_in_zecimal(gps[4], gps[3])
    return (lat, lon)

 # if __name__ == "__main__":
 #     print(gps_zecimal("probe/foto.jpg"))

# B4

def carve_zip(cale, iesire="gasit.zip"):
    date = open(cale, "rb").read()
    i = date.find(b"PK\x03\x04")
    if i == -1:
        return i
    open(iesire, "wb").write(date[i:])
    return i



if __name__ == "__main__":
    poz = carve_zip("probe/ascuns.png")
    print(carve_zip("probe/ascuns.png"))

if poz != -1:
    with zipfile.ZipFile("gasit.zip") as z:
        print(z.namelist())
        for nume in z.namelist():
            print(nume, "->", z.read(nume).decode())

# FLAG{ascuns_prin_carving}
```

Fig 3.2 – B3/B4

```python
# B5

def ascunde(cale_gazda, mesaj, cale_iesire="stego.png"):
    img = Image.open(cale_gazda).convert("RGB")
    canale = [c for px in img.getdata() for c in px]
    biti = "".join(f"{x:08b}" for x in mesaj + b"\x00")

    if len(biti) > len(canale):
        raise ValueError("Mesaj prea lung!")

    for i, bit in enumerate(biti):
        canale[i] = (canale[i] & ~1) | int(bit)

    px_nou = [tuple(canale[i:i + 3]) for i in range(0, len(canale), 3)]
    img.putdata(px_nou)
    img.save(cale_iesire)

# B6

def dezvaluie(cale):
    img = Image.open(cale).convert("RGB")
    canale = [c for px in img.getdata() for c in px]

    biti = "".join(str(c & 1) for c in canale)

    octeti = bytearray()
    for i in range(0, len(biti) - 7, 8):
        octet = int(biti[i:i + 8], 2)
        if octet == 0:
            break
        octeti.append(octet)
    return octeti.decode()
```

Fig 3.3 – B5/B6

Concluzii:

În urma realizării lucrării de laborator nr. 6, am acumulat cunoștințe practice despre metodele de codificare, decodare și ascundere a informațiilor. Utilizând Python, am aplicat tehnici precum Base64, XOR, analiza hashurilor MD5, file carving și steganografia LSB.

În cadrul lucrării au fost identificate următoarele flaguri:

1. `FLAG{straturi_de_encoding}`
2. `FLAG{xor_un_octet}`
3. `FLAG{ascuns_prin_carving}`
4. `FLAG{ascuns_in_pixeli}`